

CA1

this is how i scape from setting password everytime:

```
(base) alireza@alirezas-MacBook-Air ~ % ls .ssh
id_ed25519 id_ed25519.pub known_hosts known_hosts.old
(base) alireza@alirezas-MacBook-Air ~ % ssh-copy-id karimi@dml0
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/Users/alireza/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
karimi@dml0's password:
```

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'karimi@dml0'"
and check to make sure that only the key(s) you wanted were added.

```
(base) alireza@alirezas-MacBook-Air ~ % ssh S1apjFHbHtLq
ssh: Could not resolve hostname s1apjfhbhtlq: nodename nor servname provided, or not known
(base) alireza@alirezas-MacBook-Air ~ % ssh karimi@dml0
Linux raspberrypi-dml0 6.6.42-v8+ #1787 SMP PREEMPT Thu Jul 25 17:44:41 BST 2024 aarch64
```

```
.----- .----- .----- .-----
| .----- || .----- || .----- || .----- | | | | | | | | | | | | | |
|| ____ ||| |__ |__||| |__ ||| |__ ||
|| _ _ \ ||| | \ / _||| | _ | ||| / _ ||
|| || \ \ ||| | V | ||| | | ||| | ( _ \ ||
|| || | ||| | \ / | ||| | | _ ||| | ' _ \ ||
|| _ | _ ' / ||| | _ V | ||| | _ / ||| | \ _ ) ||
|| | _ _ ' ||| | _ | _ ||| | _ | _ ' ||
```



```
Last login: Mon Nov 3 14:48:10 2025 from 172.16.157.194
karimi@raspberrypi-dml0:~ $
```

One of the errors i got was about OOM(out of memory) and it makes me to set the `#SBATCH --mem-per-cpu=512m` in the .sh file

To sync the files (before logging into the cluster):

```
rsync -avz --progress
/Users/alireza/Documents/uni/term7/DMLS/CA/1/ karimi@dml0:~/CA1
```

I used this command for adding time into the .out file:

```
(time python pi_serial.py) &> pi_serial.out
```

To run a .sh file:

```
sbatch pi_parallel_v1_c1.sh
```

To watch what is happening:

```
squeue -u karimi
```

Q1

I got help from: <https://gemini.google.com/share/56aa2cd70669>

1.1

```
karimi@raspberrypi-dml0:~/CA1/1.1 $ cat pi_serial.out
Calculation of pi: 3.141590743731832
```

```
real 0m22.281s
user 0m22.251s
sys 0m0.028s
```

1.2

The first process (rank 0) gets all the "easy" low-`k` terms, while the last process gets all the "hard" high-`k` terms. The total time is limited by the slowest process.

c1:

```
karimi@raspberrypi-dml0:~/CA1/1.2 $ cat pi_parallel_v1_c1.out
/var/spool/slurmd/job06207/slurm_script: line 7: module: command not found
Hello from process 1 out of 2 processes
Process 1 computed its partial sum in 17.3174 seconds.
Hello from process 0 out of 2 processes
Process 0 computed its partial sum in 5.2407 seconds.
Calculation of pi: 3.1415907437318245

real 0m17.931s
user 0m0.015s
sys 0m0.023s
```

c2:

```
karimi@raspberrypi-dml0:~/CA1/1.2 $ cat pi_parallel_v1_c2.out
/var/spool/slurmd/job06208/slurm_script: line 7: module: command not found
Hello from process 1 out of 4 processes
Process 1 computed its partial sum in 3.9517 seconds.
Hello from process 0 out of 4 processes
Process 0 computed its partial sum in 1.2946 seconds.
Calculation of pi: 3.1415907437318324
Hello from process 2 out of 4 processes
Process 2 computed its partial sum in 6.8650 seconds.
```

```
Hello from process 3 out of 4 processes
Process 3 computed its partial sum in 10.2235 seconds.
```

```
real 0m10.955s
user 0m0.026s
sys 0m0.009s
```

1.3

This cyclic (or round-robin) method provides **excellent load balancing**. Each process receives an even mix of easy (low-`k`) and hard (high-`k`) terms. As a result, all processes finish at roughly the same time.

c1:

```
karimi@raspberrypi-dm10:~/CA1/1.3 $ cat pi_parallel_v2_c1.out
/var/spool/slurmd/job06209/slurm_script: line 7: module: command not found
Hello from process 1 out of 2 processes
Process 1 computed its partial sum in 11.1288 seconds.
Hello from process 0 out of 2 processes
Process 0 computed its partial sum in 11.2248 seconds.
Calculation of pi: 3.1415907437318653

real 0m11.828s
user 0m0.014s
sys 0m0.019s
```

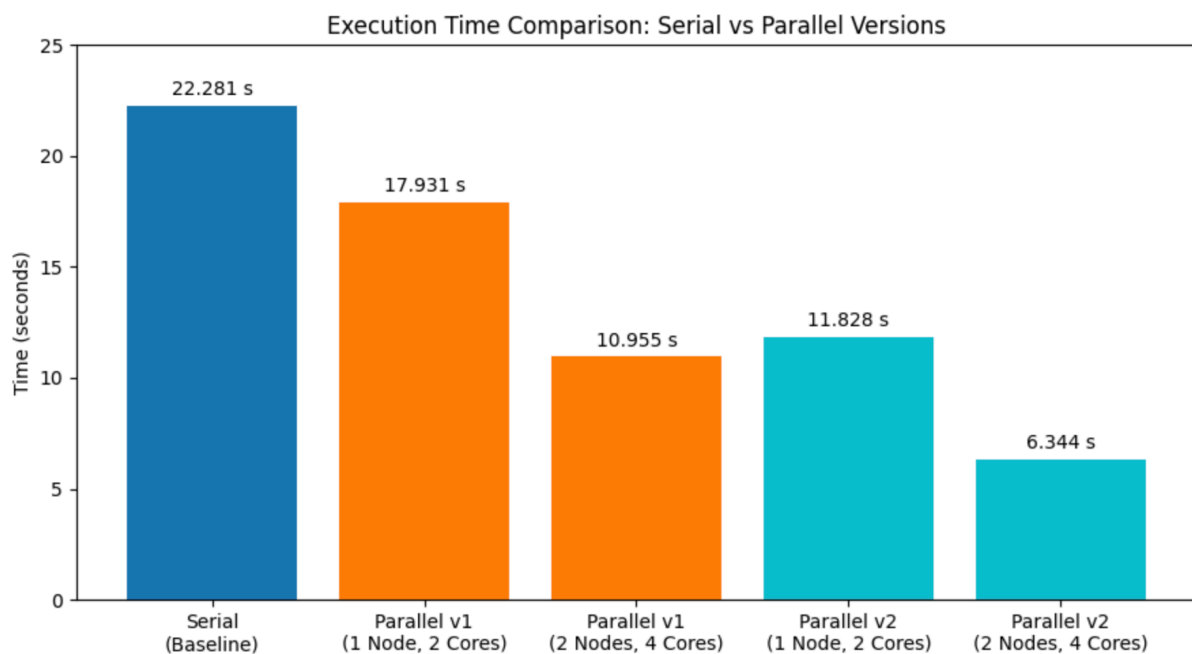
c2:

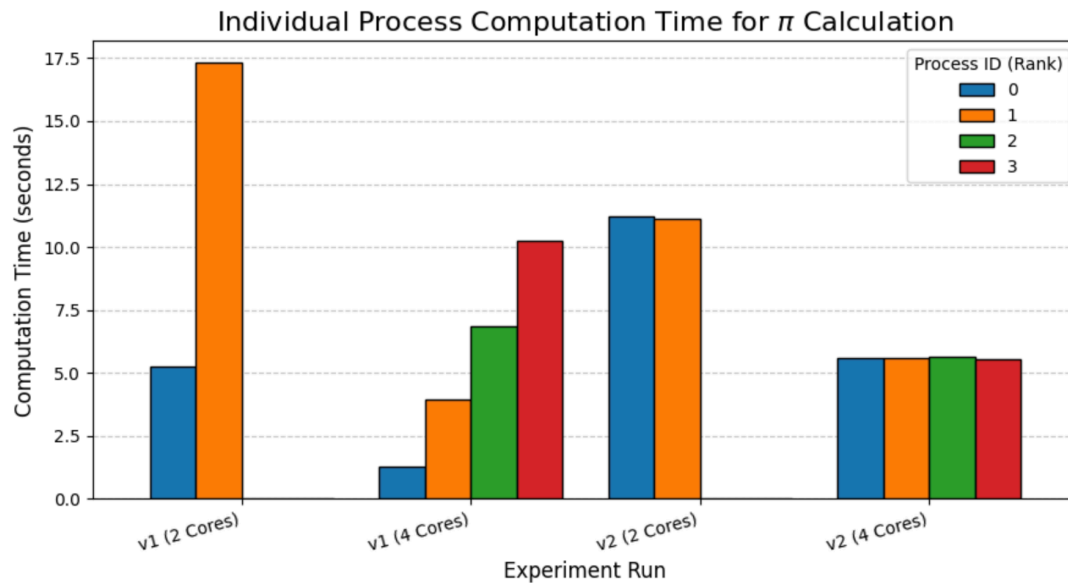
```
karimi@raspberrypi-dm10:~/CA1/1.3 $ cat pi_parallel_v2_c2.out
/var/spool/slurmd/job06210/slurm_script: line 7: module: command not found
Hello from process 3 out of 4 processes
Process 3 computed its partial sum in 5.5522 seconds.
Hello from process 1 out of 4 processes
Process 1 computed its partial sum in 5.6092 seconds.
```

Hello from process 2 out of 4 processes
Process 2 computed its partial sum in 5.6313 seconds.
Hello from process 0 out of 4 processes
Process 0 computed its partial sum in 5.5933 seconds.
Calculation of pi: 3.141590743731835

real 0m6.344s
user 0m0.025s
sys 0m0.010s

Comparison:





Q2

`fit_transform` for train and `transform` for test:

The test set (`X_test`) is supposed to simulate new, unseen data from the real world. You must not "learn" anything from it. If you were to call `fit_transform` on your `X_test` data, the scaler would learn a *new* mean and standard deviation from the test set. This is a form of **data leakage** and will make your evaluation results inaccurate.

The usage of `With`

We use it whenever we are doing calculations that are **not** part of the training's backpropagation step, such as model initialization or final evaluation. So stop tracking gradients in this block.

2.1

Result:

with scaling

```
karimi@raspberrypi-dm10:~/CA1/2.1 $ time python logreg_serial.py
Loading data from .npz files...
Error: Could not find file ../data/data1.npz
Please make sure the file paths are correct and accessible.
```

```
real 0m8.746s
user 0m8.196s
sys 0m0.973s
karimi@raspberrypi-dm10:~/CA1/2.1 $ time python logreg_serial.py
Loading data from .npz files...
All 3 datasets loaded successfully.
Total merged dataset shape: X=(100000, 50), y=(100000,)
unique labels: [0 1]
Training set size: 80000
Testing set size: 20000
```

```
Starting serial training for 30 epochs...
Epoch [5/30], Loss: 0.6909
Epoch [10/30], Loss: 0.6882
Epoch [15/30], Loss: 0.6857
Epoch [20/30], Loss: 0.6833
Epoch [25/30], Loss: 0.6809
Epoch [30/30], Loss: 0.6787
Training finished.
```

```
--- 📊 Baseline Serial Results ---
Total Training Time: 0.5466 seconds
Final Test Accuracy: 64.59%
```

```
real 0m13.429s
user 0m13.408s
sys 0m1.437s
```

without scaling:

```
karimi@raspberrypi-dm10:~/CA1/2.1 $ time python logreg_serial.py
```

```
Loading data from .npy files...
```

```
All 3 datasets loaded successfully.
```

```
Total merged dataset shape: X=(100000, 50), y=(100000,)
```

```
unique labels: [0 1]
```

```
Training set size: 80000
```

```
Testing set size: 20000
```

```
Starting serial training for 30 epochs...
```

```
Epoch [5/30], Loss: 0.6692
```

```
Epoch [10/30], Loss: 0.6551
```

```
Epoch [15/30], Loss: 0.6457
```

```
Epoch [20/30], Loss: 0.6386
```

```
Epoch [25/30], Loss: 0.6331
```

```
Epoch [30/30], Loss: 0.6286
```

```
Training finished.
```

```
--- 📊 Baseline Serial Results ---
```

```
Total Training Time: 0.5284 seconds
```

```
Final Test Accuracy: 65.85%
```

```
real 0m12.677s
```

```
user 0m13.129s
```

```
sys 0m1.515s
```

2.2

this is how i installed the `torch` library and `sklearn` . go to their website and select the linux (because the server of the cluster is using linux) and then see the commands.

NOTE: Latest Stable PyTorch requires Python 3.9 or later. Latest Preview (Nightly) PyTorch requires Python 3.10 or later.

Lightning Studios

PyTorch Build	Stable (2.9.0)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Pip	LibTorch	Source	
Language	Python		C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 13.0	CPU

Run this Command:

```
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Previous versions of PyTorch →

Don't forget to use the flag `--break-system-packages` for the course cluster.

```
karimi@raspberrypi-dm10:~/CA1/2.2 $ pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cpu
error: externally-managed-environment
```

× This environment is externally managed

↳ To install Python packages system-wide, try apt install python3-xyz, where xyz is the package you are trying to install.

If you wish to install a non-Debian-packaged Python package, create a virtual environment using python3 -m venv path/to/venv. Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make sure you have python3-full installed.

For more information visit <http://rptl.io/venv>

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this,

at the risk of breaking your Python installation or OS,
by passing `--break-system-packages`.

hint: See [PEP 668](#) for the detailed specification.

```
karimi@raspberrypi-dm10:~/CA1/2.2 $ pip3 install torch torchvision --index-url  
https://download.pytorch.org/whl/cpu --break-system-packages
```

To run the .sh file:

```
sbatch ./scenario_1.sh
```

And to see the queue every 2 sec, run:

```
watch squeue
```

The output of scenario_1.out (with scaling):

```
karimi@raspberrypi-dm10:~/CA1/2.2 $ cat scenario_1.out
```

```
--- [Rank 2] Federated Training Complete ---
```

```
--- 📊 [Rank 2] Final Results ---
```

```
Total Training Time: 0.2387 seconds
```

```
Final Test Accuracy: 51.19%
```

```
--- [Rank 3] Federated Training Complete ---
```

```
--- 📊 [Rank 3] Final Results ---
```

```
Total Training Time: 0.2388 seconds
```

```
Final Test Accuracy: 64.44%
```

```
--- Starting Federated Learning ---
```

```
Total Processes: 4
```

```
Number of Rounds: 1
```

```
Local Epochs: 10
```

```
-----
```

```

--- [Rank 0] Federated Training Complete ---

--- MASTER: Task complete. ---
the final average accuracy across workers is: 0.5612219389030548
--- [Rank 1] Federated Training Complete ---

--- 📊 [Rank 1] Final Results ---
Total Training Time: 3.6834 seconds
Final Test Accuracy: 52.74%

real 0m20.618s
user 0m0.025s
sys 0m0.009s

```

The output of scenario_2.out (with scaling):

```

karimi@raspberrypi-dm10:~/CA1/2.2 $ cat scenario_2.out
--- Starting Federated Learning ---
Total Processes: 4
Number of Rounds: 10
Local Epochs: 1
-----
--- [Rank 0] Federated Training Complete ---

--- MASTER: Task complete. ---
the final average accuracy across workers is: 0.5610719464026799
--- [Rank 2] Federated Training Complete ---

--- 📊 [Rank 2] Final Results ---
Total Training Time: 0.2572 seconds
Final Test Accuracy: 51.19%
--- [Rank 1] Federated Training Complete ---

--- 📊 [Rank 1] Final Results ---

```

```
Total Training Time: 2.0979 seconds
Final Test Accuracy: 52.71%
--- [Rank 3] Federated Training Complete ---
```

```
--- 📊 [Rank 3] Final Results ---
Total Training Time: 0.2485 seconds
Final Test Accuracy: 64.42%
```

```
real 0m17.978s
user 0m0.015s
sys 0m0.019s
```

The output of scenario_1.out (without scaling):

```
karimi@raspberrypi-dm10:~/CA1/2.2 $ cat scenario_1.out
--- Starting Federated Learning ---
Total Processes: 4
Number of Rounds: 1
Local Epochs: 10
-----
--- [Rank 0] Federated Training Complete ---

--- MASTER: Task complete. ---
the final average accuracy across workers is: 0.6243687815609219
--- [Rank 3] Federated Training Complete ---

--- 📊 [Rank 3] Final Results ---
Total Training Time: 0.1599 seconds
Final Test Accuracy: 62.16%
--- [Rank 2] Federated Training Complete ---

--- 📊 [Rank 2] Final Results ---
Total Training Time: 0.1599 seconds
```

Final Test Accuracy: 65.40%
--- [Rank 1] Federated Training Complete ---

--- 📊 [Rank 1] Final Results ---
Total Training Time: 2.7120 seconds
Final Test Accuracy: 59.76%

The output of scenario_2.out (without scaling):

```
karimi@raspberrypi-dm10:~/CA1/2.2 $ cat scenario_2.out  
--- Starting Federated Learning ---  
Total Processes: 4  
Number of Rounds: 10  
Local Epochs: 1  
-----  
--- [Rank 0] Federated Training Complete ---  
  
--- MASTER: Task complete. ---  
the final average accuracy across workers is: 0.6306684665766712  
--- [Rank 2] Federated Training Complete ---  
  
--- 📊 [Rank 2] Final Results ---  
Total Training Time: 0.2489 seconds  
Final Test Accuracy: 66.16%  
--- [Rank 3] Federated Training Complete ---  
  
--- 📊 [Rank 3] Final Results ---  
Total Training Time: 0.2441 seconds  
Final Test Accuracy: 63.07%  
--- [Rank 1] Federated Training Complete ---  
  
--- 📊 [Rank 1] Final Results ---  
Total Training Time: 2.6788 seconds
```

Final Test Accuracy: 59.97%

real 0m17.809s

user 0m0.016s

sys 0m0.030s

slurmstepd-raspberrypi-dm10: error: Timeout waiting for /sys/fs/cgroup/system.slice/raspberrypi-dm10_slurmstepd.scope/job_6201/step_batch/slurm/cgroup.events to become empty.

2.3

Malicious part:

With scaling:

```
karimi@raspberrypi-dm10:~/CA1/2.3 $ cat malicious.out
```

```
--- Starting Federated Learning ---
```

```
Total Processes: 4
```

```
Number of Rounds: 10
```

```
Local Epochs: 1
```

```
-----
```

```
--- [Rank 0] Federated Training Complete ---
```

```
--- MASTER: Task complete. ---
```

```
the final average accuracy across workers is: 0.5017249137543123
```

```
--- [Rank 2] Federated Training Complete ---
```

```
--- 📊 [Rank 2] Final Results ---
```

```
Total Training Time: 0.3063 seconds
```

```
Final Test Accuracy: 5.14%
```

```
--- [Rank 3] Federated Training Complete ---
```

```
--- 📊 [Rank 3] Final Results ---
```

```
Total Training Time: 0.3166 seconds
```

```
Final Test Accuracy: 50.08%  
--- [Rank 1] Federated Training Complete ---
```

```
--- 📊 [Rank 1] Final Results ---  
Total Training Time: 2.8680 seconds  
Final Test Accuracy: 95.29%
```

```
real 0m18.146s  
user 0m0.010s  
sys 0m0.036s
```

Without scaling:

```
karimi@raspberrypi-dm10:~/CA1/2.3 $ cat malicious.out
```

```
--- Starting Federated Learning ---
```

```
Total Processes: 4  
Number of Rounds: 10  
Local Epochs: 1
```

```
-----  
--- [Rank 0] Federated Training Complete ---
```

```
--- MASTER: Task complete. ---  
the final average accuracy across workers is: 0.5821708914554272  
--- [Rank 1] Federated Training Complete ---
```

```
--- 📊 [Rank 1] Final Results ---  
Total Training Time: 2.7364 seconds  
Final Test Accuracy: 69.49%  
--- [Rank 2] Federated Training Complete ---
```

```
--- 📊 [Rank 2] Final Results ---  
Total Training Time: 0.2465 seconds  
Final Test Accuracy: 47.16%  
--- [Rank 3] Federated Training Complete ---
```

--- [Rank 3] Final Results ---

Total Training Time: 0.2506 seconds

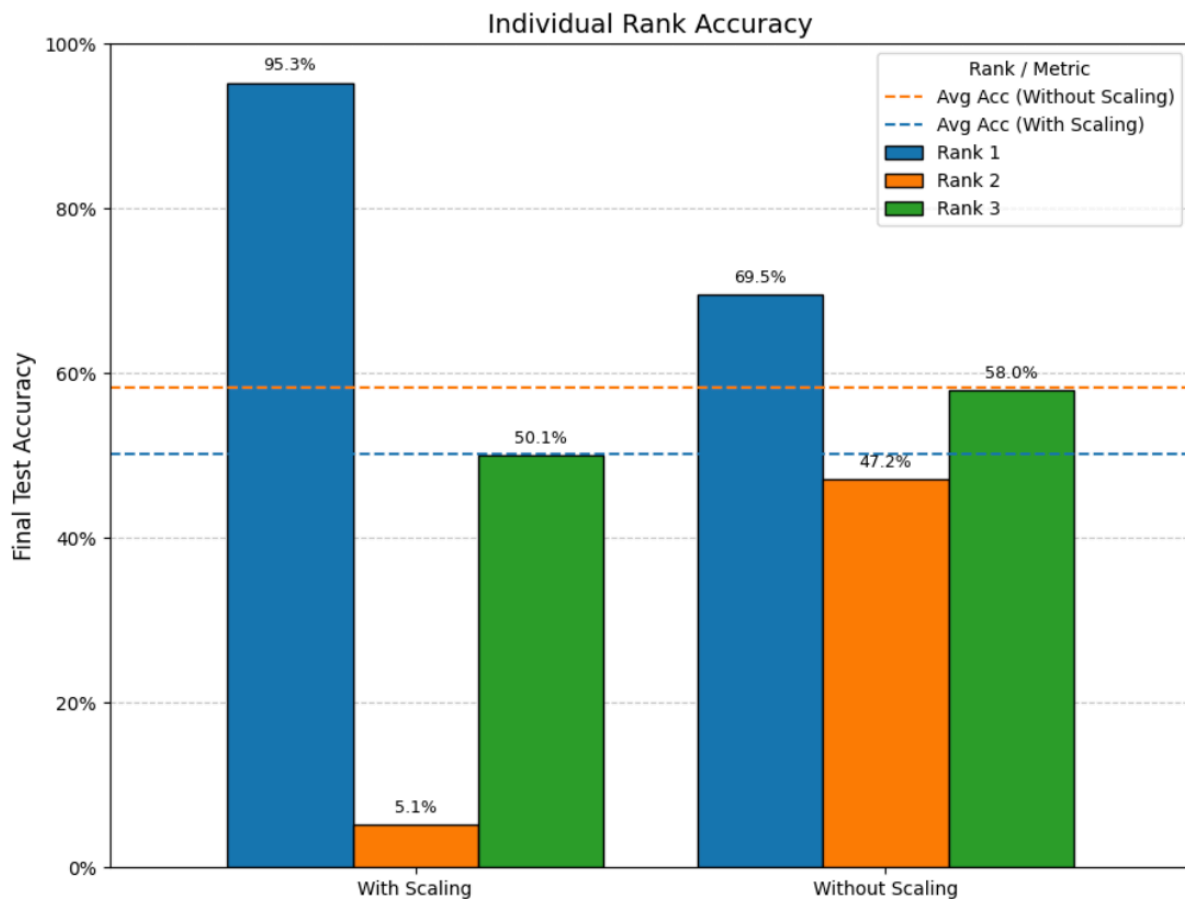
Final Test Accuracy: 58.00%

real 0m17.906s

user 0m0.023s

sys 0m0.012s

As you can see, other ranks are polluted by the rank 1, because the weight update is changing a lot in its desired direction.



This is a link for analyzing what is happening:<https://gemini.google.com/share/863784de481c>

This is a classic example of what's known as an **update scaling attack** or **model poisoning** in federated learning.

When the server averages these updates, the update from Rank 1 completely *dominates* the average. The tiny updates from Ranks 2 and 3 become statistically insignificant, like adding a few drops of water to a bucket.

Think of it this way:

- **Rank 2 update:** +1
- **Rank 3 update:** -2
- **Rank 1 update:** +500
- **Server's Average:** $(1 - 2 + 500) / 3 = 166.3$

The final average (166.3) is almost entirely determined by Rank 1's "shout" of +500.

For Rank 1, this new global model is *already* almost identical to its own local model.

and It has successfully **forced the global model to fit its local data perfectly**, ignoring the data from all other clients.

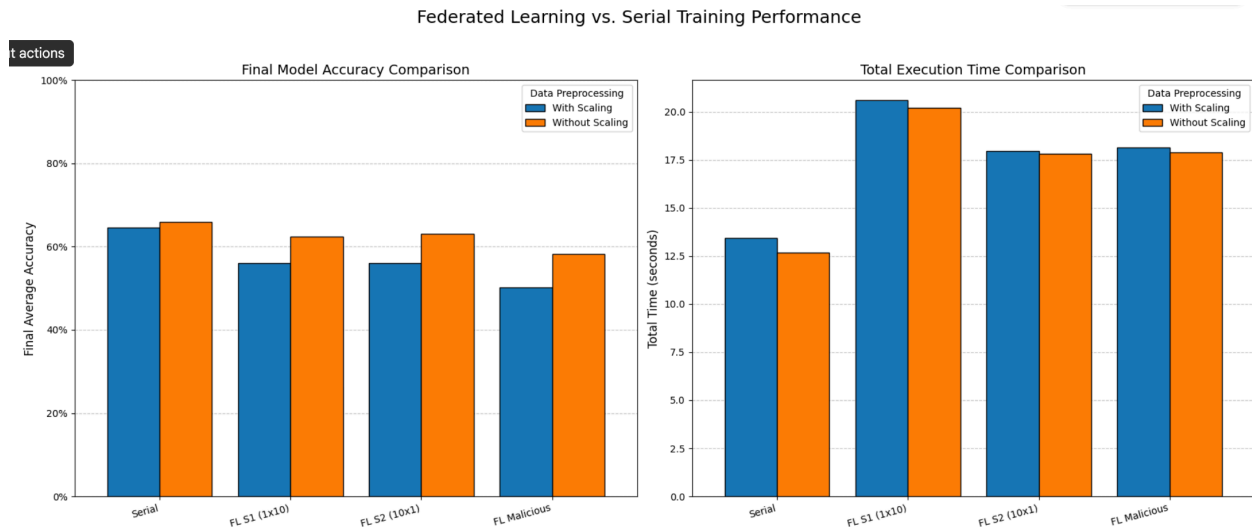
When Ranks 2 and 3 try to use this model on *their* data, it performs terribly. To make matters worse, their learning rate is so low (0.001) that they are **unable to "correct" or re-train this bad model** in their local training steps. They are effectively "stuck" with a model that is useless for them. This is why their accuracy plummets

Comparison:

Comparison plot saved as ['fl_vs_serial_comparison.png'](#)

Run	Scaling	Time (s)	Accuracy
Serial	With Scaling	13.4290	0.6459
Serial	Without Scaling	12.6770	0.6585
FL S1 (1×10)	With Scaling	20.6180	0.5612
FL S1 (1×10)	Without Scaling	20.2120	0.6244

FL S2 (10×1)	With Scaling	17.9780	0.5611
FL S2 (10×1)	Without Scaling	17.8090	0.6307
FL Malicious	With Scaling	18.1460	0.5017
FL Malicious	Without Scaling	17.9060	0.5822



Why "10 Rounds, 1 Epoch" (63.1%) is 0.7% better than "1 Round, 10 Epochs" (62.4%)

For this specific problem, the 0.7% advantage shows that frequent aggregation (more rounds) is more important than heavy local training (more epochs) to get the best result.

Why the Serial (65.8%) is ~3% better than the best Federated (63.1%)

You almost always expect the Serial/Centralized model to be the "gold standard" for accuracy. The goal of a good FL system is to get as close as possible to that benchmark, and a ~2.7% drop is often considered a very good trade-off for the massive privacy benefits of not having to collect all the data in one place. Two reason:

- **Non-IID Data:** Client data is rarely perfectly balanced. One client might have different data than another. Averaging models trained on these different data slices is statistically less efficient than training on all the data at once.
- **Aggregation Loss:** The act of averaging model weights (the "FedAvg" algorithm) is a heuristic. It's not a mathematically perfect substitute for

centralized training, and this approximation introduces a small amount of error, which results in a slightly lower accuracy.

Comparing the time:

At first, because the scaling is an extra preprocessing step, it takes some time(0.4 sec)

Then we can see that the federated learning is taking a lot more time(4-to-7 sec) it is because of MPI communication overhead.

You can see among the federated scenarios, that the S2(10 round and 1 epoch) is taking more time than S1(1 round and 10 epoch), because the communication between different ranks is more in the S2 than S1. about 10 times more communication. So it takes more time (2-3 sec)

Q3

if we run this code:

```
import numpy as np
print(np.show_config())
```

we see this:

```
(base) alireza@alirezas-MacBook-Air 3.1 % python bench_det_10k.py
blas_armpl_info:
  NOT AVAILABLE
blas_mkl_info:
  NOT AVAILABLE
blis_info:
  NOT AVAILABLE
openblas_info:
  libraries = ['openblas', 'openblas']
  library_dirs = ['/Users/alireza/anaconda3/lib']
  language = c
  define_macros = [('HAVE_CBLAS', None)]
```

```

blas_opt_info:
  libraries = ['openblas', 'openblas']
  library_dirs = ['/Users/alireza/anaconda3/lib']
  language = c
  define_macros = [('HAVE_CBLAS', None)]
lapack_armpl_info:
  NOT AVAILABLE
lapack_mkl_info:
  NOT AVAILABLE
openblas_lapack_info:
  libraries = ['openblas', 'openblas']
  library_dirs = ['/Users/alireza/anaconda3/lib']
  language = c
  define_macros = [('HAVE_CBLAS', None)]
lapack_opt_info:
  libraries = ['openblas', 'openblas']
  library_dirs = ['/Users/alireza/anaconda3/lib']
  language = c
  define_macros = [('HAVE_CBLAS', None)]
Supported SIMD extensions in this NumPy install:
  baseline = NEON,NEON_FP16,NEON_VFPV4,ASIMD
  found = ASIMDHP,ASIMDDP
  not found = ASIMDFHM
None

```

So we are using `OpenBLAS` as the default library attached with numpy.

Using `OpenBLAS` library

one of my errors:

```

(base) alireza@alirezas-MacBook-Air 3.1 % python bench_det_10k.py
/Users/alireza/anaconda3/lib/python3.11/site-packages/numpy/linalg/linalg.py:
2139: RuntimeWarning: overflow encountered in det

```

```
r = _umath_linalg.det(a, signature=signature)
Determinant of the matrix: -inf
```

This was my previous code:

```
import numpy as np
N = 4000
M = 4000
matrix = np.random.rand(N,M)
det_matrix = np.linalg.det(matrix)
print(f"Determinant of the matrix: {det_matrix}")
```

The Problem: Numerical Limits with the `np.linalg.det` function

The Solution: Use `np.linalg.slogdet`

- **What it returns:** It returns two values:

1. `sign` : The sign of the determinant ($+1$, -1 , or 0).
2. `logdet` : The natural logarithm ($\ln$) of the *absolute value* of the determinant.

The actual determinant is $\text{sign} \times e^{\text{logdet}}$. You almost always just use the `logdet` value in calculations.

This is the new code:

```
import numpy as np

N = 10000
M = 10000

matrix = np.random.rand(N, M)

sign, logdet = np.linalg.slogdet(matrix)
```

```
print(f"\nThe determinant is (theoretically): {sign} * e^({logdet:.2f})")
```

And this command to add to the output:

```
(time python bench_det_10k.py) &> bench_det_10k.txt
```

The result was:

```
The determinant is (theoretically): 1.0 * e^(28630.38)
python bench_det_10k.py 63.39s user 3.74s system 399% cpu 16.822 total
```

which means it takes 16.822 sec

running the 20k matrix was really time consuming, so I preferred to compare the 5k and 8k.

Since I am using Mac m1 (Apple Silicon), I can use `Accelerate` library rather than `OpenBLAS`

Using `Accelerate` library

I download and install the `Miniforge3-MacOSX-arm64.sh` installer

and then this command:

```
conda install -c conda-forge "numpy" "libblas=*accelerate"
```

This tells Conda to install NumPy and specifically requires the `libblas` (the BLAS library) dependency to be the `accelerate` version.

This does not work, so let's do the second way:

To get NumPy to use **Apple's Accelerate framework**, you must follow one of the methods I mentioned previously.

The Conda "solving environment" freeze you saw earlier happened because your `(base)` environment is very complex. The **best and safest solution** is to create a new, clean environment specifically for this.

Here are the correct steps again:

1. **Create a new, clean environment:** Bash

```
conda create -n np_accel -c conda-forge
```

2. **Activate it:** Bash

```
conda activate np_accel
```

3. Install NumPy with Apple Accelerate: Bash

(Inside the `np_accel` environment, this command will be very fast).

```
conda install -c conda-forge "numpy" "libblas=*accelerate"
```

4. **Verify your success:** Bash

```
python -c "import numpy; numpy.show_config()"
```

You should now see `accelerate` and `vecLib` in the output, confirming you are using Apple's native framework.

I got help from this link:

<https://gemini.google.com/share/03763ef75437>

As you can see, I am using `np_accel` now:

```
Preparing transaction: done
Verifying transaction: done
```

Executing transaction: **done**

```
(np_accel) alireza@alirezas-MacBook-Air ~ % python -c "import numpy; num
py.show_config()"
/Users/alireza/anaconda3/envs/np_accel/lib/python3.14/site-packages/nump
y/_config__py:155: UserWarning: Install `pyyaml` for better output
  warnings.warn("Install `pyyaml` for better output", stacklevel=1)
{
  "Compilers": {
    "c": {
      "name": "clang",
      "linker": "ld64",
      "version": "19.1.7",
      "commands": "arm64-apple-darwin20.0.0-clang",
      "args": "-ftree-vectorize, -fPIC, -fstack-protector-strong, -O2, -pipe, -isys
tem, /Users/alireza/anaconda3/envs/np_accel/include, -fdebug-prefix-map=/
Users/runner/miniforge3/conda-bld/bld/rattler-build_numpy_1761161593/work
=/usr/local/src/conda/numpy-2.3.4, -fdebug-prefix-map=/Users/alireza/anac
onda3/envs/np_accel=/usr/local/src/conda-prefix, -DACCELERATE_NEW_LAP
ACK, -D_FORTIFY_SOURCE=2, -isystem, /Users/alireza/anaconda3/envs/np_a
ccel/include, -mmacosx-version-min=11.0, -mmacosx-version-min=11.0",
      "linker args": "-Wl,-headerpad_max_install_names, -Wl,-dead_strip_dylibs,
-Wl,-rpath,/Users/alireza/anaconda3/envs/np_accel/lib, -L/Users/alireza/anac
onda3/envs/np_accel/lib, -ftree-vectorize, -fPIC, -fstack-protector-strong, -O
2, -pipe, -isystem, /Users/alireza/anaconda3/envs/np_accel/include, -fdebug-
prefix-map=/Users/runner/miniforge3/conda-bld/bld/rattler-build_numpy_1761
161593/work=/usr/local/src/conda/numpy-2.3.4, -fdebug-prefix-map=/Users/
alireza/anaconda3/envs/np_accel=/usr/local/src/conda-prefix, -DACCELERAT
E_NEW_LAPACK, -D_FORTIFY_SOURCE=2, -isystem, /Users/alireza/anaconda
3/envs/np_accel/include, -mmacosx-version-min=11.0, -mmacosx-version-mi
n=11.0"
    },
    "cython": {
      "name": "cython",
      "linker": "cython",
      "version": "3.1.5",
      "commands": "cython"
```

```

},
"clang": {
  "name": "clang",
  "linker": "ld64",
  "version": "19.1.7",
  "commands": "arm64-apple-darwin20.0.0-clang++",
  "args": "-ftree-vectorize, -fPIC, -fstack-protector-strong, -O2, -pipe, -stdlib=libc++, -fvisibility-inlines-hidden, -fmessage-length=0, -isystem, /Users/alireza/anaconda3/envs/np_accel/include, -fdebug-prefix-map=/Users/runner/miniforge3/conda-bld/bld/rattler-build_numpy_1761161593/work=/usr/local/src/conda/numpy-2.3.4, -fdebug-prefix-map=/Users/alireza/anaconda3/envs/np_accel=/usr/local/src/conda-prefix, -DACCELERATE_NEW_LAPACK, -D_FORTIFY_SOURCE=2, -isystem, /Users/alireza/anaconda3/envs/np_accel/include, -mmacosx-version-min=11.0, -mmacosx-version-min=11.0",
  "linker args": "-Wl,-headerpad_max_install_names, -Wl,-dead_strip_dylibs, -Wl,-rpath,/Users/alireza/anaconda3/envs/np_accel/lib, -L/Users/alireza/anaconda3/envs/np_accel/lib, -ftree-vectorize, -fPIC, -fstack-protector-strong, -O2, -pipe, -stdlib=libc++, -fvisibility-inlines-hidden, -fmessage-length=0, -isystem, /Users/alireza/anaconda3/envs/np_accel/include, -fdebug-prefix-map=/Users/runner/miniforge3/conda-bld/bld/rattler-build_numpy_1761161593/work=/usr/local/src/conda/numpy-2.3.4, -fdebug-prefix-map=/Users/alireza/anaconda3/envs/np_accel=/usr/local/src/conda-prefix, -DACCELERATE_NEW_LAPACK, -D_FORTIFY_SOURCE=2, -isystem, /Users/alireza/anaconda3/envs/np_accel/include, -mmacosx-version-min=11.0, -mmacosx-version-min=11.0"
},
},
"Machine Information": {
  "host": {
    "cpu": "arm64",
    "family": "aarch64",
    "endian": "little",
    "system": "darwin"
  },
  "build": {
    "cpu": "aarch64",
    "family": "aarch64",

```

```

    "endian": "little",
    "system": "darwin"
  },
  "cross-compiled": true
},
"Build Dependencies": {
  "blas": {
    "name": "blas",
    "found": true,
    "version": "3.9.0",
    "detection method": "pkgconfig",
    "include directory": "/Users/alireza/anaconda3/envs/np_accel/include",
    "lib directory": "/Users/alireza/anaconda3/envs/np_accel/lib",
    "openblas configuration": "unknown",
    "pc file directory": "/Users/alireza/anaconda3/envs/np_accel/lib/pkgconfi
g"
  },
  "lapack": {
    "name": "lapack",
    "found": true,
    "version": "3.9.0",
    "detection method": "pkgconfig",
    "include directory": "/Users/alireza/anaconda3/envs/np_accel/include",
    "lib directory": "/Users/alireza/anaconda3/envs/np_accel/lib",
    "openblas configuration": "unknown",
    "pc file directory": "/Users/alireza/anaconda3/envs/np_accel/lib/pkgconfi
g"
  }
},
"Python Information": {
  "path": "/Users/alireza/anaconda3/envs/np_accel/bin/python",
  "version": "3.14"
},
"SIMD Extensions": {
  "baseline": [
    "NEON",

```

```

    "NEON_FP16",
    "NEON_VFPV4",
    "ASIMD"
  ],
  "found": [
    "ASIMDHP"
  ],
  "not found": [
    "ASIMDFHM"
  ]
}
}
(np_accel) alireza@alirezas-MacBook-Air ~ %

```

Each time, to activate the `Accelerate`, I should do this :

```
conda activate np_accel
```

Then I can run using the `Accelerate` library.

And at the end, to deactivate the `Accelerate` and get back to previous settings and use the `OpenBLAS` again, i do this:

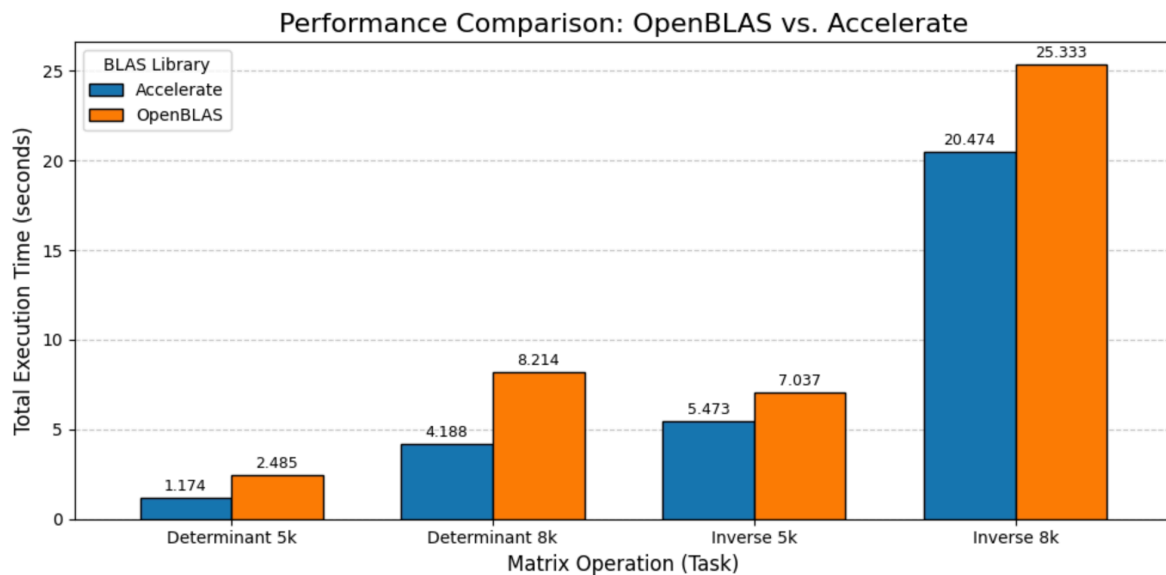
```
conda deactivate
```

Results:

Now I compare the `OpenBLAS` and `Accelerate` libraries in calculating the determinant and inversion of matrix of size 5000 and 8000: (time applied to the whole code)

Task	Library	Time (s)
Determinant 5k	OpenBLAS	2.485
Determinant 5k	Accelerate	1.174

Determinant 8k	OpenBLAS	8.214
Determinant 8k	Accelerate	4.188
Inverse 5k	OpenBLAS	7.037
Inverse 5k	Accelerate	5.473
Inverse 8k	OpenBLAS	25.333
Inverse 8k	Accelerate	20.474



Analysis of why Apple's **Accelerate** framework is outperforming **OpenBLAS**.

The primary reason for this is **deep hardware integration**. OpenBLAS is a fantastic, general-purpose, open-source library, but Apple's Accelerate is a proprietary library built specifically to extract the maximum performance from their own hardware.

Accelerate: Apple's framework has exclusive access to the **Apple Matrix Coprocessor (AMX)**. This is a specialized piece of silicon *within the M-series chip* designed for one purpose: performing matrix multiplication and other linear algebra operations at incredible speeds.

- **Hardware-Specific Tuning:** Accelerate's routines are hand-tuned by Apple's engineers to perfectly match the cache sizes, memory bandwidth, and

instruction pipelines of each specific M-series chip (M1, M2, M3, etc.).

- **Efficient Threading:** The framework includes its own highly efficient threading model (using Grand Central Dispatch - GCD) that is optimized for the performance and efficiency cores of Apple Silicon. It knows exactly how to schedule tasks to avoid bottlenecks, whereas OpenBLAS uses a more general-purpose threading model.
-

- **Determinant:** For a large matrix, the most efficient way to find the determinant is to first perform an **LU Decomposition**. This process (which is very computationally expensive) is the vast majority of the work. Once it's done, the determinant is just the product of the diagonal elements, which is incredibly fast. Accelerate uses AMX for this. When you ask it to perform an LU decomposition, it offloads the most intense parts of that calculation (the GEMM (General Matrix-Matrix Multiplication operations) directly to this super-fast, specialized AMX hardware.

| Determinant $\approx$ 99% (LU Decomposition) + 1% (Simple Multiplication)

- **Matrix Inverse:** This operation *also* starts with the **exact same LU Decomposition**. However, after that's done, it still has a lot of work to do. It must perform a series of "triangular solves" (forward and backward substitutions) to find the final inverse matrix.

| Inverse $\approx$ 70% (LU Decomposition) + 30% (Triangular Solves)(Note: These percentages are just for illustration)